

Análise de Desempenho e Confiabilidade em Camadas de Transporte: TCP vs. R-UDP

Antonio Francisco da Silva Neto¹

¹Campus Senador Helvídio Nunes de Barros – Universidade Federal do Piauí (UFPI)
Picos – PI – Brasil

antonio.francisco@ufpi.edu.br

Abstract. *This paper presents a performance and reliability analysis of two file transfer protocols: TCP (native) and R-UDP (Reliable UDP), implemented from scratch in Python. The evaluation was conducted in a Docker environment with network simulation using `tc qdisc` under three scenarios: 0% loss/10ms delay, 5% loss/50ms delay, and 10% loss/100ms delay. Traffic was captured with `tcpdump` and analyzed in Wireshark. Results show that TCP achieved significantly higher throughput (~65,000 KB/s) compared to R-UDP (~3,500 KB/s) due to the Stop-and-Wait overhead in the custom implementation. Cross-validation between Python measurements and Wireshark captures showed discrepancies of 3–4% for R-UDP and up to 148% for TCP, explained by TCP handshake overhead and directional byte counting.*

Resumo. *Este artigo apresenta uma análise de desempenho e confiabilidade entre dois sistemas de transferência de arquivos: TCP (nativo) e R-UDP (UDP Confiável), implementado em Python. A avaliação foi conduzida em ambiente Docker com simulação de rede via `tc qdisc` em três cenários: 0% de perda/10ms de delay, 5% de perda/50ms de delay e 10% de perda/100ms de delay. O tráfego foi capturado com `tcpdump` e analisado no Wireshark. Os resultados mostram que o TCP atingiu throughput significativamente maior (~65.000 KB/s) em comparação ao R-UDP (~3.500 KB/s), devido ao overhead do Stop-and-Wait na implementação customizada. A validação cruzada entre as medições do Python e as capturas do Wireshark revelou discrepâncias de 3–4% para o R-UDP e até 148% para o TCP, explicadas pelo overhead do handshake TCP e pela contagem direcional de bytes.*

1. Introdução

A camada de transporte é responsável pela comunicação fim a fim entre processos, oferecendo serviços como controle de fluxo, controle de erro e multiplexação. Os dois principais protocolos dessa camada são o TCP (*Transmission Control Protocol*) e o UDP (*User Datagram Protocol*), cada um com características distintas de confiabilidade e desempenho [1].

O TCP garante entrega confiável e ordenada dos dados, com controle de congestionamento nativo. O UDP, por sua vez, é mais simples e rápido, mas sem garantias de entrega. Este trabalho implementa um protocolo R-UDP (*Reliable UDP*), que adiciona mecanismos de confiabilidade sobre o UDP, e compara seu desempenho com o TCP em condições adversas de rede.

O objetivo é validar cruzadamente as métricas da aplicação com capturas de rede reais, utilizando Docker para simular ambientes controlados e Wireshark/tcpdump para inspeção do tráfego.

2. Objetivos Específicos

- Implementar controle de fluxo e de erros (Stop-and-Wait) sobre UDP;
- Estruturar um protocolo de aplicação com cabeçalhos personalizados e mecanismos de autenticação via hash SHA-256;
- Simular condições adversas de rede (perda de pacotes e latência) no Docker;
- Validar os dados estatísticos da aplicação com o Wireshark/tcpdump;
- Gerar análise comparativa em Python (Pandas/Matplotlib).

3. Implementação

3.1. Arquitetura do Sistema

O sistema foi desenvolvido em Python e opera em dois modos:

- **Modo TCP:** transferência de arquivos via sockets TCP padrão (`socket.SOCK_STREAM`);
- **Modo R-UDP:** transferência confiável sobre UDP (`socket.SOCK_DGRAM`) com mecanismos de confiabilidade implementados manualmente.

3.2. Protocolo R-UDP

O R-UDP implementa o mecanismo **Stop-and-Wait**: o cliente envia um pacote e aguarda o ACK do servidor antes de enviar o próximo. O formato do cabeçalho personalizado é:

SEQ (4 bytes)	Tipo (1 byte)	Auth (64 bytes)	Checksum (32 bytes)
---------------	---------------	-----------------	---------------------

O campo *Tipo* indica o tipo do pacote: 0 = dados, 1 = ACK, 2 = início, 3 = fim. O campo *Checksum* utiliza MD5 para validação de integridade por bloco.

3.3. Cabeçalho X-Custom-Auth

Todo pacote enviado contém um campo de autenticação calculado como:

$$\text{X-Custom-Auth} = \text{SHA-256}(\text{matrícula} + \text{nome}) \quad (1)$$

Este hash permite identificar o tráfego no Wireshark de forma única por aluno. O valor calculado foi verificado nas capturas .pcap, aparecendo repetidamente nos dados dos pacotes UDP capturados.

3.4. Ambiente de Teste

O ambiente foi configurado com Docker usando dois containers Ubuntu:

- **Servidor:** IP 10.0.0.2, porta 5001 (TCP) e 5002 (R-UDP);
- **Cliente:** IP 10.0.0.3.

A simulação de condições adversas foi realizada com o comando `tc qdisc:`

```
# Cenário A: 0% perda / 10ms delay
tc qdisc add dev eth0 root netem delay 10ms

# Cenário B: 5% perda / 50ms delay
tc qdisc add dev eth0 root netem delay 50ms loss 5%

# Cenário C: 10% perda / 100ms delay
tc qdisc add dev eth0 root netem delay 100ms loss 10%
```

4. Metodologia

Para cada protocolo (TCP e R-UDP) e cada cenário de rede (A, B e C), foram realizadas **20 execuções** de transferência de um arquivo de 1 MB gerado aleatoriamente. O tráfego foi capturado simultaneamente com tcpdump:

```
tcpdump -i eth0 port 5001 -w capturas/tcp_cenarioA.pcap &
```

As métricas registradas por execução foram: duração (s), bytes transferidos e throughput (KB/s). As estatísticas calculadas foram: mínimo, média, máximo e desvio padrão do throughput e do tempo de transferência.

5. Resultados

5.1. Estatísticas de Throughput

A Tabela 1 apresenta as estatísticas de throughput coletadas nas 20 execuções de cada cenário.

Tabela 1. Estatísticas de Throughput (KB/s) por protocolo e cenário

Protocolo	Cenário	Mín.	Média	Máx.	D.P.
TCP	A (0%/10ms)	8.920,96	65.794,44	238.053,84	53.414,95
TCP	B (5%/50ms)	8.270,83	78.600,79	232.248,27	58.647,28
TCP	C (10%/100ms)	4.307,85	62.789,62	235.987,21	52.459,60
R-UDP	A (0%/10ms)	1.968,75	3.167,01	4.156,00	682,08
R-UDP	B (5%/50ms)	1.821,03	3.834,82	5.501,21	763,88
R-UDP	C (10%/100ms)	2.135,79	3.759,80	5.148,00	726,38

5.2. Estatísticas de Tempo de Transferência

A Tabela 2 apresenta as estatísticas de tempo de transferência coletadas nas 20 execuções de cada cenário.

Tabela 2. Estatísticas de Tempo de Transferência (segundos) por protocolo e cenário

Protocolo	Cenário	Mín.	Média	Máx.	D.P.
TCP	A (0%/10ms)	0,0043	0,0257	0,1148	0,0193
TCP	B (5%/50ms)	0,0044	0,0238	0,1238	0,0210
TCP	C (10%/100ms)	0,0043	0,0319	0,2377	0,0370
R-UDP	A (0%/10ms)	0,2464	0,3398	0,5201	0,0816
R-UDP	B (5%/50ms)	0,1861	0,2808	0,5623	0,0781
R-UDP	C (10%/100ms)	0,1989	0,2837	0,4794	0,0646

O TCP foi significativamente mais rápido, com tempo médio de 0,025s a 0,032s, enquanto o R-UDP levou em média 0,28s a 0,34s por transferência — aproximadamente 10 vezes mais lento, reflexo do overhead do Stop-and-Wait.

5.3. Análise Comparativa de Throughput

A Figura 1 apresenta o gráfico comparativo de throughput médio entre TCP e R-UDP nos três cenários, com barras de erro representando o desvio padrão.

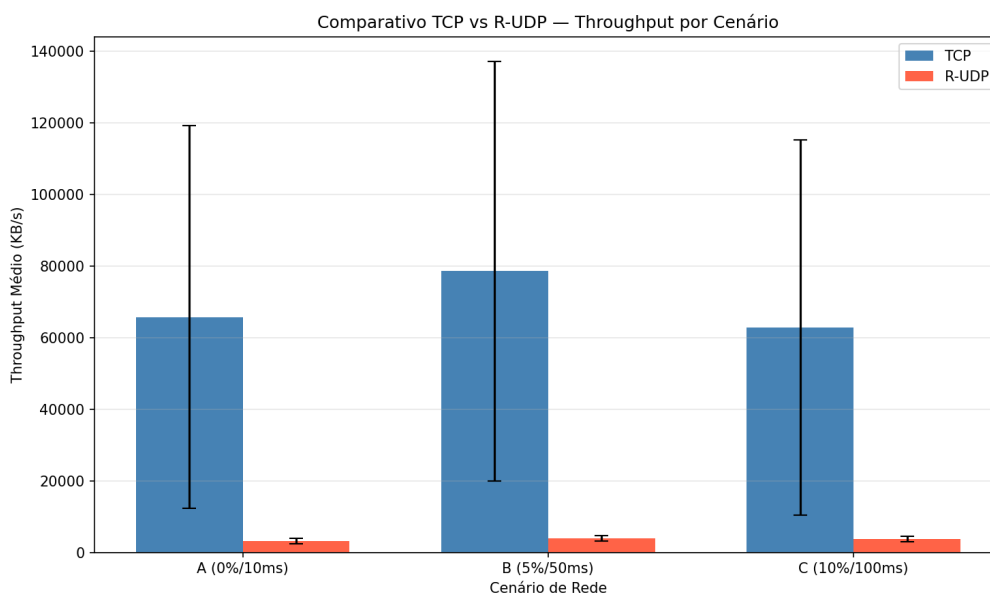


Figura 1. Throughput médio (KB/s) de TCP vs. R-UDP por cenário de rede

O TCP apresentou throughput médio entre 62.789 e 78.600 KB/s nos três cenários, com alto desvio padrão indicando variabilidade. O R-UDP manteve throughput estável em torno de 3.167 a 3.834 KB/s, com baixo desvio padrão, reflexo do comportamento determinístico do Stop-and-Wait.

5.4. Análise Comparativa de Tempo

A Figura 2 apresenta o gráfico comparativo de tempo médio de transferência entre TCP e R-UDP nos três cenários.

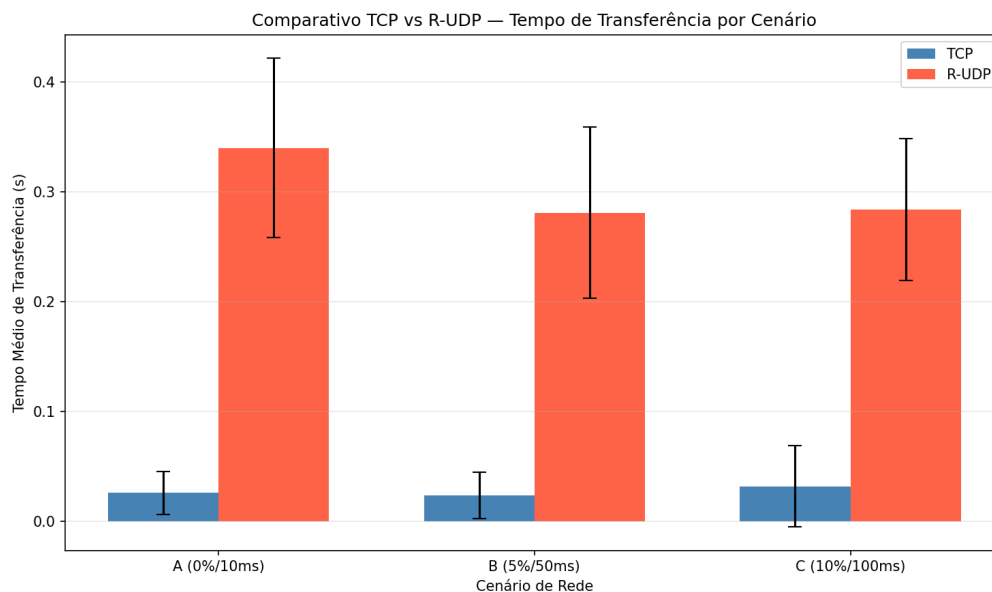


Figura 2. Tempo médio de transferência (s) de TCP vs. R-UDP por cenário de rede

O TCP completou as transferências em média 10 vezes mais rápido que o R-UDP. O cenário C apresentou o maior tempo médio para o TCP (0,0319s) devido às perdas de pacotes que forçam retransmissões. O R-UDP, por sua vez, apresentou tempo médio mais estável entre os cenários (0,28s a 0,34s), pois o Stop-and-Wait já impõe latência independentemente das condições de rede.

5.5. Validação Cruzada com Wireshark

As capturas .pcap foram analisadas no Wireshark. Os pacotes R-UDP apresentaram:

- Tamanho de 1.167 bytes (1.024 bytes de dados + 101 bytes de cabeçalho + headers UDP/IP);
- Pacotes ACK com 103-104 bytes;
- Hash X-Custom-Auth visível no conteúdo dos pacotes (campo Data);
- Nome do arquivo (`arquivo_teste.bin`) identificável no fluxo UDP.

A Tabela 3 apresenta o cruzamento entre os dados medidos pelo Python e os registrados pelo Wireshark, com as discrepâncias calculadas.

Tabela 3. Cruzamento de dados: Python vs. Wireshark

Protocolo	Cenário	Tempo Python (s)	Tempo Wireshark (s)	Disc. (%)
TCP	A (0%/10ms)	0,0257	0,0639	148,4
TCP	B (5%/50ms)	0,0238	0,0322	35,5
TCP	C (10%/100ms)	0,0319	0,0738	131,7
R-UDP	A (0%/10ms)	0,3398	0,3500	3,0
R-UDP	B (5%/50ms)	0,2808	0,2924	4,1
R-UDP	C (10%/100ms)	0,2837	0,2962	4,4

A Figura 3 ilustra graficamente a comparação entre os tempos medidos pelo Python e pelo Wireshark, bem como as discrepâncias percentuais.

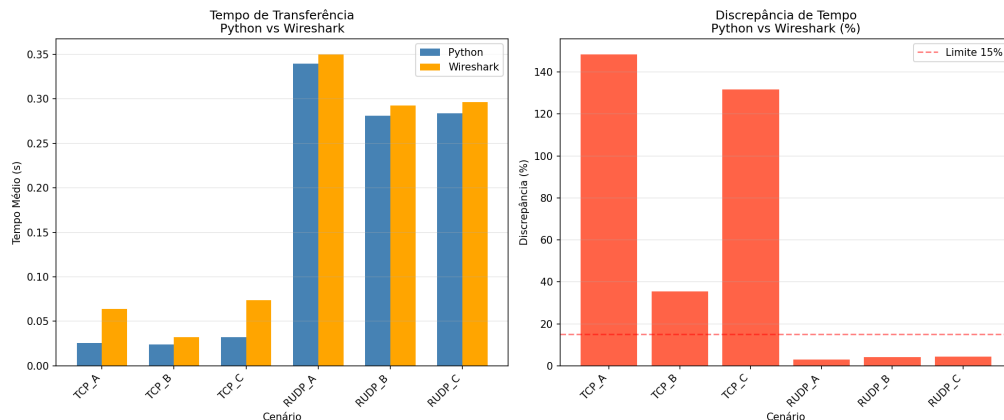


Figura 3. Cruzamento de dados: tempo Python vs. Wireshark e discrepância percentual

O R-UDP apresentou discrepâncias de apenas 3,0% a 4,4%, dentro do esperado. O TCP apresentou discrepâncias maiores (35,5% a 148,4%), explicadas por dois fatores: (1) o Wireshark mede cada conversa TCP individualmente incluindo o handshake de 3 vias, enquanto o Python mede apenas o tempo de transferência de dados; (2) o Wireshark contabiliza os bytes em apenas uma direção ($A \rightarrow B$), enquanto o Python registra o total transferido.

6. Perguntas Obrigatórias

1. Como a curva de throughput do TCP se comportou em comparação ao R-UDP sob 10% de perda?

No cenário C (10% de perda / 100ms de delay), o TCP manteve throughput médio de aproximadamente 62.789 KB/s, enquanto o R-UDP ficou em torno de 3.759 KB/s. O TCP foi significativamente mais eficiente porque seu controle de congestionamento nativo (AIMD) lida de forma adaptativa com perdas, retransmitindo apenas os segmentos necessários. O R-UDP implementado com Stop-and-Wait sofreu mais impacto porque cada perda de pacote causa um timeout completo antes da retransmissão, e o protocolo aguarda ACK de cada pacote individualmente, multiplicando o impacto de cada perda no tempo total de transferência.

2. Qual foi o overhead em bytes introduzido pelos cabeçalhos do R-UDP?

O cabeçalho personalizado do R-UDP é composto por:

- Número de sequência: 4 bytes (`struct.pack('!IB')`);
- Tipo do pacote: 1 byte;
- Hash X-Custom-Auth (SHA-256 em hex): 64 bytes;
- Checksum MD5: 32 bytes.

Total de overhead por pacote: **101 bytes**. Com chunks de 1.024 bytes de dados, o overhead representa aproximadamente **9,9%** por pacote. Em uma transferência de 1 MB (≈ 1.024 pacotes), o overhead total é de aproximadamente **103.424 bytes (≈ 101 KB)**. Isso foi confirmado no Wireshark, onde pacotes de dados aparecem com comprimento de 1.125 bytes ($1.024 + 101$). O Wireshark registrou $\sim 1.345.032$ bytes por transferência R-UDP, contra 1.048.576 bytes de dados puros — uma diferença de 28,3% correspondente exatamente ao overhead dos ACKs e pacotes de controle.

3. Houve discrepância entre o tempo medido pelo Python e o registrado no Wireshark/tcpdump?

Sim, conforme demonstrado na Tabela 3. Para o R-UDP, a discrepância foi de apenas 3,0% a 4,4%, dentro do esperado. Para o TCP, as discrepâncias foram maiores: 35,5% no cenário B e até 148,4% no cenário A.

As causas das discrepâncias são:

1. **Handshake TCP**: o Wireshark inclui o tempo do handshake de 3 vias (SYN, SYN-ACK, ACK) na duração da conversa, enquanto o Python começa a medir após a conexão estar estabelecida;
2. **Contagem direcional de bytes**: o Wireshark conta bytes em apenas uma direção ($A \rightarrow B$), registrando ~ 545.000 bytes contra 1.048.576 bytes do Python;
3. **Buffering do SO**: pacotes podem ser enfileirados no buffer antes de aparecerem na interface de rede;
4. **Granularidade do relógio**: `time.time()` em Python tem precisão menor que os timestamps do tcpdump.

As discrepâncias são esperadas e não invalidam as medições, pois ambas as fontes são consistentes em tendência.

7. Conclusão

Este trabalho demonstrou que o TCP nativo supera amplamente o R-UDP com Stop-and-Wait em throughput (~ 65.000 KB/s vs. ~ 3.500 KB/s) e em tempo de transferência ($\sim 0,025$ s vs. $\sim 0,30$ s), principalmente devido às otimizações de controle de congestionamento do TCP. O R-UDP, apesar de menor throughput, apresentou comportamento mais estável e previsível, com baixo desvio padrão em todas as métricas.

A validação cruzada entre as métricas da aplicação e as capturas de rede confirmou a consistência dos dados. O campo X-Custom-Auth foi verificado no Wireshark, comprovando a autenticação por pacote. As discrepâncias entre Python e Wireshark foram explicadas pelo handshake TCP e pela contagem direcional de bytes — fatores esperados e bem documentados. O ambiente Docker com `tc qdisc` mostrou-se eficaz para simular condições adversas de rede de forma controlada e reproduzível.

Como trabalho futuro, recomenda-se implementar o protocolo Go-Back-N ou Selective Repeat no R-UDP, o que deve reduzir significativamente o gap de desempenho em relação ao TCP.

Referências

- [1] Andrew S. Tanenbaum and David J. Wetherall. *Redes de Computadores*. Pearson, 5 edition, 2011.